



OMERTA AI

Owner's Manual

Silence Is The Only Unbreakable Code

A coding, systems and firmware agent that runs on your own hardware and asks before it acts. Android without Termux, Linux, macOS, Windows.

You supply the model; OMERTA supplies everything around it.

Version 1.9.1

read chapter 2 first - it explains what you must supply

CHAPTER 1

What OMERTA AI Is

OMERTA AI is an agent that runs on hardware you own. It reads and writes your code, drives a real shell, inspects and builds firmware images, remembers what you teach it, and asks your permission before it changes anything.

The one-sentence version

It is the whole machine around a language model — the memory, the tools, the terminal, the sandbox, the approval gate, the interface — running entirely on your device, with the model itself being a part you choose and plug in.

What makes it different

- **It asks first.** Every action the model wants to take is classified and shown to you before it happens. Destructive and irreversible operations always ask, no matter how you set the policy.
- **It runs on your hardware.** The Android build carries its own Python interpreter. There is no Termux, no companion app, no bootstrap download, and no server of ours in the path.
- **It has a real terminal.** A genuine PTY-backed shell with job control, plus 288 bundled commands, because Android's own userland is too thin to work in.
- **It does not guess.** The firmware side in particular labels every value CONFIRMED, LIKELY, INFERRED or UNKNOWN, and refuses to turn an UNKNOWN into a guess to keep a task moving.
- **It is yours to shape.** Projects, branched and queued chats, passcode-locked private chats, a learning shelf you can add to and take away from, and a theme system down to the fonts and the icon.

Who it is for

Someone who wants an agent on a phone or a laptop that behaves like a capable colleague rather than a toy: one that can be handed a real repository, a real device tree or a real shell, and that will tell you what it does not know instead of inventing it.

CHAPTER 2

The Model — Read This First

The single most important thing to understand about OMERTA: it does not contain a language model, and it will not think without one.

OMERTA AI is not itself an AI model. It is an agent — the harness. Claude, GPT and the rest are models: the part that reasons. OMERTA is everything else. Installed and opened with nothing configured, it will start, show its interface, run its terminal and browse your files, and it will not answer a single question, because there is nothing behind the chat box yet.

So what do I have to supply?

Exactly one of these two things:

1. An API key, for a model in the cloud

Paste it into Settings. The key is stored on your device and sent only to that provider. This is the easiest route and gives you the strongest models, at the cost of needing a connection and of your prompts leaving the device.

Provider	Setting	Notes
Claude (Anthropic)	ANTHROPIC_API_KEY	Sonnet and Opus; the default choice
OpenAI	OPENAI_API_KEY	GPT and o-series
OpenRouter	OPENROUTER_API_KEY	Many models behind one key
Groq	GROQ_API_KEY	Very fast, open-weight models
Google Gemini	GEMINI_API_KEY	Gemini via its OpenAI-compatible endpoint

2. A local model server, for no key and no internet

OMERTA speaks to Ollama, llama.cpp and LM Studio. Point it at the host and port in Settings. Nothing leaves your network, there is no key, no per-message cost and no usage limit.

Server	Default address	Getting started
Ollama	http://127.0.0.1:11434	ollama serve && ollama pull qwen2.5-coder:7b
llama.cpp	http://127.0.0.1:8080	llama-server -m your-model.gguf
LM Studio	http://127.0.0.1:1234/v1	Enable the local server in its UI

The honest caveat on phones

Those local addresses say 127.0.0.1 — this device. On a laptop that is easy: run Ollama on the same machine. **On a phone there is currently nothing to run**, because OMERTA does not yet bundle an inference engine. So on Android, "offline mode" today means *pointing at a model server on your own network* — your PC at 192.168.x.x running Ollama — not a model running on the handset.

That is the one gap between what OMERTA is and the phrase "runs on its own without an API key". Closing it means shipping a llama.cpp server inside the APK and letting you load a GGUF file. It is the first recommendation in Chapter 12.

Online, offline and auto

Mode	What it does
online	Cloud providers only. Never falls back to a local model.
offline	Local providers only. Never touches the network — verified by test, not merely intended.
auto	Prefers your chosen provider and falls back through the routing order.

CHAPTER 3

Installing

One agent, five shapes. Pick whichever matches the machine in front of you; they share the same code, the same approval gate and the same data format.

Android

Copy the APK to the phone and open it; allow "install unknown apps" for whatever you opened it from. Everything is inside: CPython, the agent, the web interface and the command-line toolset. It needs no network to start.

Updates install straight over the previous version and keep your chats, projects, themes and learned files. **Do not uninstall first** — that deletes all of it.

Desktop

Format	Install
ApplImage	<code>chmod +x OMERTA-AGENT-desktop-*.ApplImage && ./OMERTA-AGENT-desktop-*.ApplImage</code>
.deb	<code>sudo apt install ./omerta-agent-desktop*_amd64.deb</code>
tar.gz	Unpack anywhere and run the binary inside. No install step.

Command line

Format	Install	Needs
CLI ApplImage	<code>chmod +x</code> and run	nothing — carries CPython 3.12
Standalone binary	<code>chmod +x omerta-linux-x86_64</code>	nothing
.deb (small)	<code>sudo apt install ./omerta-agent*_all.deb</code>	system python3
pip	<code>pip install omerta_agent-*.whl</code>	Python 3.9+ (incl. Termux)

Use `apt install ./file.deb` rather than `dpkg -i` for the small .deb: dpkg alone will not pull python3-requests and python3-yaml, and leaves the package unconfigured.

Confirming which build you are running

Every build carries a stamp naming the commit that produced it. In the app it is the box at the top of Settings — tap to copy. On a terminal, `omerta --version`. Over HTTP, `GET /api/version`. If a copy is unstamped it says so rather than claiming a version it cannot prove.

CHAPTER 4

The Approval Gate

The rule that everything else is built around: the model proposes, you dispose.

Every action the agent wants to take — run a command, write a file, flash an image — is classified before it happens and shown to you with the exact command it intends to run. Nothing is paraphrased.

The four tiers

Tier	Examples	Behaviour
DENY	<code>rm -rf /</code> , wiping a disk, fork bombs	Refused outright. Cannot be approved, because the mistake is unrecoverable and it is nearly always a typo.
HIGH RISK	flashing firmware, force push, git reset --hard, deleting a branch, package removal	Always asks, on every policy setting, including the most permissive.
NORMAL	building, running tests, editing files, committing	Asks, unless you have set the policy to trust this class.
LOW RISK	reading files, listing directories, status queries	Runs without interrupting you.

The policy dial

In Settings, or the chip at the top of the screen. It moves where the line sits for NORMAL actions only — **DENY** and **HIGH RISK** do not move, whatever you choose.

Setting	Meaning
asks first	The default. Anything that changes state asks.
build trust	Routine build and test commands run; edits still ask.
trusted	Normal work proceeds; high-risk and destructive still ask.

What the gate does not cover, and why

The terminal, the editor's saves, the sandbox and loading a file by path do not prompt. Those are *you* operating your own device, not the model acting. Prompting for each keystroke would be absurd. That is also exactly why those four surfaces refuse any request that did not come from this device — see Chapter 10.

CHAPTER 5

The Interface

Six tabs. The same interface on the phone, the desktop app and any browser on your network.

CHAT

Where the work happens. No message limit and no truncation of your conversation. The agent will argue with you: if it thinks your approach is wrong it will say so and offer an alternative — and then do what you decided.

Work modes change how it approaches a request:

Mode	Behaviour
plan	Produces a plan and stops. Writes nothing.
brainstorm	Options and trade-offs, deliberately divergent.
research	Reads and reports. Investigates without changing anything.
build	Executes, still through the approval gate.

TERMINAL

A real shell with a real controlling terminal — job control, Ctrl-C, and full-screen programs such as vi all work. The banner tells you the shell, whether it is a PTY, whether the deny-guard is on, and how many tools are available. See Chapter 6.

CODE

A file browser and editor over your project, with search across the indexed tree. Edits are written straight to disk and a backup of the previous contents is kept.

CHATS

- **Projects** — separate workspaces. Each has its own memory, its own learned documents and its own conversation history.
- **Branching** — fork a conversation at any message to try a different direction without losing the original.
- **Queue** — line up several messages; each runs as the previous one finishes, so you can leave it working.
- **Private chats** — locked behind a passcode and encrypted on disk with a key derived from it. Forget the passcode and the contents are gone; there is no recovery, which is the point.
- **Archive, export and import** — conversations are yours to move between devices.

LEARN

Upload documents, code or notes for the agent to study. It indexes them and uses them as background for that project. Crucially you can **remove** what it learned — individual documents and individual learned behaviours — and the removal is real: the row is deleted and verified gone, not merely hidden.

SETTINGS

Provider and model, network mode, approval policy, API keys, the terminal guard, sandbox containment, appearance, and the build stamp.

CHAPTER 6

The Terminal and Its Toolset

Android gives an app a very thin userland. OMERTA brings its own.

`/system/bin` on Android is `toybox`, and it is missing most of what a terminal is actually for: no `wget`, no `awk`, no `vi`, no `less`, no `unzip`, no `tar` that can write. So the APK ships a static BusyBox and puts **288 commands** on your `PATH`.

What you get

`wget tar unzip gzip bzip2 xz cpio sed awk grep find xargs diff patch vi less more sort uniq cut tr wc head tail nc ping traceroute nslookup ip ifconfig netstat route ps top kill df du stat strings xxd sha256sum md5sum base64 bc dc tree watch crond blkid fdisk losetup mount` — and roughly 240 more.

A full list is written to `T00LS.txt` beside the tool directory on the device; read it with `less ../T00LS.txt` from the `bin` directory.

Deliberate omissions

Not linked	Why
<code>su</code> , <code>login</code> , <code>passwd</code>	They need <code>setuid</code> root. On a rooted phone, linking these would shadow the real <code>/system/bin/su</code> and break root — use that one. On an unrooted phone there is no working <code>sudo</code> at all, and shipping one that always fails would be worse than not shipping it.
<code>sh</code> , <code>ash</code>	The session shell is chosen deliberately elsewhere. Run <code>ash</code> by name if you want BusyBox's.
<code>init</code> , <code>reboot</code> , <code>halt</code> , <code>chroot</code> , <code>insmod</code>	Require privileges no Android app has.

Two limits worth knowing

HTTPS is not authenticated. BusyBox's TLS does not validate certificates and will tell you so. `wget https://...` works, but treat it as unauthenticated transport: fine for fetching something you are going to checksum, not fine for anything secret. Ask the agent to fetch instead when it matters — it uses Python's TLS, which does validate.

There is no python3 on the command line. The app embeds CPython in its own process rather than as a separate executable, so there is no interpreter for the shell to run. Use the `CODE` tab, or ask the agent to run Python for you.

The deny-guard

The hard-deny list applies in the terminal too: `rm -rf /` is refused even though you typed it yourself. Matching is argument-precise, so `rm -rf /tmp/build` is not caught by the rule against `rm -rf /`. Set `OMERTA_TERM_GUARD=0` if you genuinely need one. Every command is logged either way.

CHAPTER 7

The Sandbox

Somewhere to run something you do not trust yet.

A scratch sandbox builds and executes code exactly as it would for real, but nothing it produces reaches your project until you say so. You can keep the result, throw it away, or keep part of it.

Two different protections

Layer	Protects	How
Scratch workspace	your project from your experiment	Writes are confined to the sandbox directory and merged only on your confirmation.
Device containment	your device from a hostile program	bubblewrap namespaces: no network by default, a deny-by-default filesystem, and a cleared environment so a sandboxed process cannot read your API keys.

Know what your device can actually enforce

Containment depends on kernel features that not every device has. OMERTA reports what yours can genuinely provide — **strict**, **relaxed** or **limits only** — and a request for more containment than the device can enforce is capped at what is real, and says so. It will not tell you a process is contained when it is not.

CHAPTER 8

Firmware Work

The part of OMERTA with the strictest rules, because the cost of being wrong is a brick.

OMERTA can unpack and rebuild boot, vendor_boot and dtbo images, read and decompile device trees, handle Android sparse images and take apart a super.img. What governs all of it is an evidence rule.

Evidence labels

Label	Meaning
CONFIRMED	Extracted from the actual firmware or kernel in front of it.
LIKELY	Strongly supported by evidence, not directly read.
INFERRED	Deduced from related facts. Treat as a hypothesis.
UNKNOWN	Not established. It stays UNKNOWN.

An UNKNOWN is never promoted to a guess to keep a task moving. If a value cannot be extracted, OMERTA tells you what it would need in order to know, rather than supplying a plausible number.

Standing rules

- The only copy of stock firmware is never modified. Work happens on a copy.
- A newly built boot image is not flashed immediately — it is inspected, compared and explained first.
- Flashing is a HIGH RISK action and always asks, on every policy setting.
- Every irreversible step states what it will do, to which partition, and what recovery looks like if it fails.

CHAPTER 9

Memory, Learning and Forgetting

What it keeps, where it keeps it, and how to make it let go.

Three kinds of memory

Kind	What it is	Scope
Conversation	The current chat, unbounded and untruncated	One chat
Facts & preferences	Things you told it to remember	Project, or shared
Learned documents	Files you uploaded in the LEARN tab	Project

Forgetting is real

Removing a fact, a document or a learned behaviour deletes it and confirms the deletion. This is worth stating because it was once broken: the store returned a status string instead of the row identifier, so every removal silently did nothing while reporting success. It is now tested, including that a deletion propagates to your other devices and is not resurrected by the next sync.

Multi-device sync

Devices exchange memory directly — phone to laptop to workstation — with no server in between. Safety travels with it: a denial recorded on one device is inherited by the others, so a refusal you set on your phone is not quietly absent on your laptop. Re-syncing is idempotent, malformed bundles are rejected rather than merged, and oversized ones are refused before they are parsed.

CHAPTER 10

Privacy and Security

What leaves your device, what cannot reach it, and what to be careful about.

What leaves the device

Situation	What is sent, and where
Cloud provider selected	Your prompt and the relevant context, to that provider only. Nothing to us — there is no "us" in the path.
Local provider selected	Nothing leaves your network.
Offline mode	Nothing leaves the device at all. Verified by a test that fails if the network is touched.

Loopback and the local-only rule

The server binds to loopback. If you expose it on your network, a token is required — and the token check cannot be fooled by a forged X-Forwarded-For, X-Real-IP or Forwarded header.

Four surfaces refuse *any* request that did not originate on this device, even with a valid token: the terminal, the editor's writes, the sandbox runner and loading a file by absolute path. They do not route through the approval gate, because they are you operating your own machine — which is precisely why they may not be driven from another one.

This was not theoretical. Each of the three was found by exploiting it: a remote caller ran `id` as root through the sandbox endpoint, overwrote a file through the editor endpoint, and read `/etc/hostname` through the learning endpoint. All three are closed, the fix is enforced in both servers immediately after authentication, and the regression is tested.

Keys and private chats

- API keys are stored on the device and never written into logs, commits or exports.
- A sandboxed process gets a cleared environment, so it cannot read your keys even if it goes looking.
- Private chats are encrypted with a key derived from your passcode (PBKDF2-HMAC-SHA256, then AES-GCM). The passcode is not stored. Lose it and the chat is unreadable — by anyone, including you.

CHAPTER 11

Troubleshooting

The things most likely to go wrong, and what they mean.

Symptom	Cause and fix
Chat does nothing, or says no provider is configured	No model is connected. See Chapter 2 — you need an API key or a local model server.
"offline mode needs a local model"	You are in offline mode with no local server reachable. Start Ollama or llama.cpp and set its address, or switch to online.
Terminal warns about job control	Fixed in 1.3.0. If you still see it, you are running an older build — check the stamp at the top of Settings.
A command is "not found" in the terminal	Check the banner shows a tool count. If it says the toolset is unavailable it gives the reason. Note there is no python3, adb or sudo — see Chapter 6 and Chapter 12.
wget warns about TLS certificates	Expected. BusyBox's TLS does not validate them. Ask the agent to fetch instead when it matters.
A .deb installs but is "unconfigured"	Use <code>apt install ./file.deb</code> , not <code>dpkg -i</code> , so dependencies are pulled.
An update lost my chats	The app was uninstalled rather than updated, or the new build was signed with a different key. Install over the top; never uninstall.
"Backend didn't come up"	The launch screen now names the reason underneath. Tap RETRY & SHOW DIAGNOSTICS : it re-runs the launch (which often succeeds on its own) and, if it fails again, prints the device, ABI, payload state and full Python traceback, with COPY and SHARE buttons. Send that text — it is the whole diagnosis, no cable or logcat needed.
First launch sits on "unpacking Python" for minutes	Expected once, on slower or 32-bit devices: the app is writing a full Python standard library to private storage. The file counter moves the whole time. It never repeats until you update the app.
Sandbox says containment is limited	Your kernel lacks the namespaces. The report is accurate — it is telling you the truth rather than overstating protection.

CHAPTER 12

What Is Missing, and What Should Come Next

An honest list. These are the gaps between what OMERTA is today and what it is trying to be, in the order they are worth closing.

1. An inference engine in the APK

The biggest gap by far. Today the phone needs either an API key or a model server elsewhere on your network. Bundling a llama.cpp server — packaged the same way the BusyBox binary is — and letting you load a GGUF file would make OMERTA genuinely self-contained: no key, no internet, no second machine. A 3B model fits comfortably on a modern handset. Everything needed to do this has now been proved to work; it is the single change that would most alter what OMERTA is.

2. A real python3 on the command line

Same mechanism, second use. A standalone CPython as a native binary would give the terminal a genuine interpreter, and with it pip, scripts, and most of what people actually open a terminal on a phone to do.

3. git and ssh

The agent understands git deeply, but the terminal has no git binary. Static builds of git, OpenSSH and curl-with-real-TLS would close the three most obvious remaining gaps in the toolset — and curl would fix the certificate-validation caveat in Chapter 6.

4. adb over TCP

USB-attached devices are out of reach without Android's USB Host API and a user-granted permission, so a plain adb binary cannot work. But `adb connect host:5555` over the network is an ordinary TCP protocol and is implementable. libimobiledevice has the same USB problem with no TCP equivalent, so it is the one item on your list that is genuinely out of reach on an unrooted phone.

5. Signing the release properly

Builds are currently signed with the Android debug key, which is what keeps updates installing over each other. Before you publish anything, generate a real release keystore, back it up in more than one place, and sign with it. **If you lose that key you can never update the app again** — every existing install would have to be removed and reinstalled, losing user data. This is the single most irreversible decision in shipping an Android app.

Smaller things worth having

- **Voice input** — dictating to an agent on a phone is far more natural than typing at it.
- **A share target** — send a file, a URL or a code snippet to OMERTA from any other app.
- **Scheduled and triggered runs** — a task that wakes on a schedule, or when a repository changes.
- **An encrypted backup** of everything (chats, projects, learned files, themes) to a single file you control.
- **Per-project approval policy** — trusted inside a scratch project, strict against production.

- **A cost meter** for cloud providers, so a long agent run cannot surprise you.
- **Model presets** — a cheap fast model for routine work and a strong one for hard problems, switched per task rather than globally.
- **Tablet and landscape layout** — the interface currently assumes a phone.

CHAPTER 13

Reference

Settings, endpoints and files.

Environment settings

Variable	Effect
OMERTA_MODE	online offline auto
OMERTA_PROVIDER	Which provider to use, or auto
OMERTA_APPROVAL_POLICY	ask build trusted
OMERTA_TERM_GUARD	0 disables the terminal deny-guard
OMERTA_ISOLATE_NET	Allow network inside the sandbox
OMERTA_LOCAL_MODEL	Model name for Ollama
OMERTA_OLLAMA_HOST	Address of your Ollama server
OMERTA_LLAMACPP_HOST	Address of your llama.cpp server
OMERTA_DATA_DIR	Where chats, memory and settings live
OMERTA_TOKEN	API token when serving beyond loopback

HTTP endpoints

Endpoint	Purpose
GET /api/version	Which build this is — version, commit, build time
GET /api/status	Providers, policy, mode, toolset, memory, skills
POST /api/chat	One turn, through the approval gate
/api/term/*	Terminal. Local-only.
/api/ws/*	Editor and workspace. Writes are local-only.
/api/scratch/*	Sandbox. Local-only.
/api/learn/*	Learning shelf. Loading by path is local-only.

Command line

Command	Does
omerta	Interactive agent in the terminal
omerta serve	Start the server and web interface

Command	Does
omerta doctor	Check the installation and report problems
omerta sync <host:port>	Sync memory with another device
omerta --version	Version, commit and build time

Where your data lives

Platform	Path
Android	/data/data/com.omerta.agent/files/data
Linux / macOS	~/.omerta (or \$OMERTA_DATA_DIR)
Windows	%USERPROFILE%\\.omerta

OMERTA AI v1.9.1 — Owner's Manual. Generated from the repository, so the version on the cover is the version of the build that produced it. Silence Is The Only Unbreakable Code.